

✓ A. Homework

Some exercises to be solved in Python...

SOLUTIONS ARE NOT UNIQUE, so be creative and do it yourself.

✓ 1. Load Excel spreadsheet

Load the file [../datasets/students.xlsx](#) which contains a table with body measures from 200 people. Set the column `Name` as index.

Display the table to understand its structure.

```
import pandas as pd
df = pd.read_excel('/content/sample_data/students.xlsx', index_col='Name')
#Establecemos la columna Name como index con index_col

print(df.info) #Mostramos información básica del DataFrame
df.head() #Mostramos las primeras filas
```

```
<bound method DataFrame.info of
Name
James      M      AU  179.342668  79.935886
John       M      BG  164.955502  65.851122
Robert     M      FI  171.523428  69.582601
Michael    M      MV  166.210330  67.333376
William    M      US  166.446515  72.379703
...
Rose       F      DE  158.146533  61.140314
Isabella   F      GR  169.051451  71.096523
Alexis     F      DK  167.521331  59.512411
Kayla      F      BG  161.200252  62.546094
Charlotte  F      SI  163.287711  63.263214
```

[200 rows x 4 columns]>

	Gender	Country	Height	Weight
Name				
James	M	AU	179.342668	79.935886
John	M	BG	164.955502	65.851122
Robert	M	FI	171.523428	69.582601
Michael	M	MV	166.210330	67.333376
William	M	US	166.446515	72.379703

Pasos siguientes: [New interactive sheet](#)

2. Table summary

Calculate the count, mean, standard deviation, minimum, maximum and quartiles of **Height** and **Weight**.

```
#Con describe podremos devolver un resumen descriptivo con todo lo que se no
print(df['Height'].describe())
print(df['Weight'].describe())
```

```
count    200.000000
mean     169.087153
std       5.674803
min       157.421492
25%      164.709325
50%      168.230444
75%      172.768739
max       186.793584
Name: Height, dtype: float64
count    200.000000
mean      69.495476
std       6.459158
min       56.909779
25%       64.666933
```

```

50%      68.597025
75%      73.341741
max       90.777996
Name: Weight, dtype: float64

```

3. Load country codes

Load the file `../datasets/country_codes.xlsx` which contains a table with country names and their *alpha-2* and *alpha-3* codes.

Display the table to understand its structure.

```
df2 = pd.read_excel('/content/sample_data/country_codes.xlsx')
```

```
print(df2.info) #Mostramos información básica del DataFrame
```

```
df2.head() #Mostramos las primeras filas
```

```

<bound method DataFrame.info of          Country name Alpha-2 code Alpha-3 code
0      Afghanistan      AF      AFG
1         Albania      AL      ALB
2         Algeria      DZ      DZA
3  American Samoa      AS      ASM
4         Andorra      AD      AND
..          ...      ...      ...
244  Western Sahara      EH      ESH
245         Yemen      YE      YEM
246         Zambia      ZM      ZMB
247        Zimbabwe      ZW      ZWE
248   Åland Islands      AX      ALA

```

```
[249 rows x 3 columns]>
```

	Country name	Alpha-2 code	Alpha-3 code
0	Afghanistan	AF	AFG
1	Albania	AL	ALB
2	Algeria	DZ	DZA
3	American Samoa	AS	ASM
4	Andorra	AD	AND

Pasos siguientes: [New interactive sheet](#)

4. Set index, drop column

Use column `Alpha-2 code` as index. Drop `Alpha-3 code` since it won't be used.

```

#Vamos a establecer el indice con set_index
df2.set_index('Alpha-2 code', inplace=True)
#inplace = True hace que modifiquemos el df original

```

```
df2.drop(columns=['Alpha-3 code'], inplace=True)
```

```
df2.head() #Mostramos las primeras filas, para verificar que se han aplicado
```

	Country name
AF	Afghanistan
AL	Albania
DZ	Algeria
AS	American Samoa
AD	Andorra

Pasos siguientes: [New interactive sheet](#)

5. Translate country codes in table people into full country names

Merge/join the two tables together in order to add a new column with the country names.

```
# Con Merge juntamos los data frames
#Funciona muy parecido a un Join en sql
df = df.merge(df2[['Country name']], left_on='Country', right_index=True,
              how='left')
#df2[['Country name']]: Selececcionamos solo la columna que necesitamos
#left_on='Country': Especifica la columna del DataFrame izquierdo (df) que
#right_index=True: Le dice a pandas que la key para unir en el df2 está en el índice
#how='left': Indica que se trata de una union de tipo left
df.head()
```

	Gender	Country	Height	Weight	Country name
Name					
James	M	AU	179.342668	79.935886	Australia
John	M	BG	164.955502	65.851122	Bulgaria
Robert	M	FI	171.523428	69.582601	Finland
Michael	M	MV	166.210330	67.333376	Maldives
William	M	US	166.446515	72.379703	United States of America (the)

Pasos siguientes: [New interactive sheet](#)

6. Select the people from Germany

```
#Imprime el dataframe con la condicion de que el nombre del pais sea igual a
estudiantes_alemanes = df[df['Country name'] == 'Germany']
print(estudiantes_alemanes)
```

Name	Gender	Country	Height	Weight	Country name
Adam	M	DE	173.867841	80.421687	Germany
Dylan	M	DE	176.604982	75.773395	Germany
Ashley	F	DE	165.295126	70.354539	Germany
Carol	F	DE	165.385105	64.171022	Germany
Rose	F	DE	158.146533	61.140314	Germany

7. Calculate average weight by country

```
avg_by_country = df.groupby('Country name')['Weight'].mean()
print(avg_by_country.head())
#De esta manera calculamos la media del peso y la agrupamos por paises
```

Country name	Weight
Australia	71.221834
Austria	68.447042
Belgium	70.356313
Brazil	69.134570
Bulgaria	72.746911

Name: Weight, dtype: float64

8. Calculate minimum and maximum values for height by country

Display minimum and maximum heights per country. Results must appear in a single table.

```
df.groupby('Country name')['Height'].agg(['min', 'max'])
#El agg aplica ambas funciones de agregacion tanto el minimo como el maximo
```

	min	max	
Country name			
Australia	164.046006	180.041598	
Austria	162.560890	178.513876	
Belgium	158.176498	182.807218	
Brazil	162.303334	179.183013	
Bulgaria	161.200252	184.491849	
Canada	163.553921	183.985524	
Cyprus	164.521192	184.611704	
Denmark	157.421492	178.211492	
Egypt	160.683908	178.242983	
Finland	161.972224	175.415478	
France	162.640605	173.977675	
Germany	158.146533	176.604982	

Greece 163.482033 186.793584

9. Calculate averages by country and gender

Israel 161.817559 172.492903

Let's calculate the average by country and gender at the same time.

Italy 163.048389 180.169017

9.1. Group by country and gender together

Japan 162.061111 178.939351

Group by Country name and Gender in the same `groupby()`, to get aggregated values in two levels

Mexico 163.562243 178.811368

Peru 164.111553 173.582104

```
df.groupby(['Country name', 'Gender'])[['Height', 'Weight']].mean()
#Agrupamos las filas en Country name y Gender y calculamos la media
#IMPORTANTE: Country name y Gender pasan a ser index
```



		Height	Weight	
Country name		Gender		
Australia		F	166.115366	65.816832
		M	175.250293	76.626835
Austria		F	165.985284	66.820252
		M	170.249126	70.399189
Belgium		F	163.740903	65.964075
		M	176.877330	76.944669
Brazil		F	164.350914	65.171183
		M	172.919351	72.531759
Bulgaria		F	161.200252	62.546094
		M	172.409542	73.880335

9.2. Pivot table

Let's pivot the table to display countries in columns and gender in rows.

```
#Antes de pivotar las tablas habrá que resetear el indice para que vuelvan a
#Ya que en el ej anterior los establecimos como indices
avg_by_country_gender = df.groupby(['Country name', 'Gender'])[['Height', 'W
height_pivot = avg_by_country_gender.pivot(index='Gender', columns='Country
print(height_pivot)

weight_pivot = avg_by_country_gender.pivot(index='Gender', columns='Country
print(weight_pivot)
```

Country name	Australia	Austria	Belgium	Brazil	Bulgaria	\
Gender		M	173.207082	74.156751		
F	166.115366	165.985284	165.35740903661608429	164.350914	161.200252	
M	175.250293	170.249126	176.877330	172.919351	172.409542	
		M	169.651108	68.510507		
Country name	Canada	Cyprus	Denmark	Egypt	Finland	...
Gender	Germany	F	162.942255	65.221958		...
F	165.963163	165.640937	163.484991	164.723127	165.809190	...
M	176.072907	174.301523	175.236411	78.097541	173.207082	...
	Greece	F	166.153299	65.840920		
Country name	Germany	Greece	Israel	Italy	Japan	\
Gender		M	172.458522	72.746025		
F	162.942255	166.153299	165.741921	166.591915	165.494684	
M	175.236411	172.458522	165.741921	76.61721184	171.381137	
		M	171.926079	70.440498		
Country name	Maldives	Mexico	Peru	Slovenia	\	
Gender	Italy	F	166.591915	67.257145		
F	160.961005	167.384661	166.260726	166.076955		
M	172.653170	172.517996	172.952268	169.639065		
		M	175.484144	76.612765		
Country name	United States of America					
Gender	Japan	F	165.494684	64.580990		

Gender		M	171.381137	70.822419					
F			163.512503						
M	Maldives	F	171.689642	60.5	63.257183				
[2 rows x 21 columns]		M	172.653170	72.377366					
Country name	Australia	Austria	Belgium	Brazil	Bulgaria	\			
Gender	Mexico	F	167.384661	66.505234					
F	65.816832	66.820252	65.954975	65.171183	62.546094				
M	76.626835	70.399189	76.944669	72.672608	73.880335				
	Peru	F	166.260726	67.086462					
Country name	Canada	Cyprus	Denmark	Egypt	Finland	...	\		
Gender		M	172.952268	73.827875		...			
F	66.323326	63.619883	64.564780	63.730551	66.393035	...			
M	Slovenia	F	166.078955	65.170817	74.156751	...			
		M	169.639065	68.938806					
Country name	Germany	Greece	Israel	Italy	Japan	\			
Gender	United States of America (the)	F	163.512503	63.608576					
F	65.221958	65.840920	66.721119	67.257145	64.580990				